

AtomicMoore — Build Plan & Setup Guide

A hands-on, explained walkthrough. Every step says **what** to run and **why** you're doing it, so you understand the stack while it gets built. Commands assume **Windows + PowerShell** and Node 18+ (check with `node -v`). Run them from `D:\coding\project\workout\`.

Context

`D:\coding\project\workout\atomic-moore-mockup\atomic-moore-mockup.html` is a static, fully-styled mockup of a workout tracker called **AtomicMoore** — 11 mobile screens plus desktop (sidebar) views. The goal is to turn that mockup into a real, **mobile-first responsive** web app.

Decisions confirmed:

- **Auth:** email/password **and** Google OAuth.
- **Routines:** fully user-editable programs/days/exercises, **seeded** with the mockup's 5 day-types (PUSH, PULL, LEGS, TORSO A, LOWER+ARMS B) as a ready-to-edit example.
- **Scope:** working **MVP** first; harden to production later if the product proves out.

The stack, and why each piece is here:

Layer	Tool	Why
Frontend	React + TypeScript	Component UI; TS catches mistakes before runtime
Styling	TailwindCSS	The mockup is utility-style CSS already — Tailwind maps to it 1:1, mobile-first by default
Build/dev	Vite	Instant dev server + fast builds for React
Server state	TanStack Query	Caching, loading/error states, refetch — so you don't hand-roll fetch logic
Backend	Express + TypeScript	Minimal, well-known HTTP API framework
Validation	Zod	One schema validates request bodies and gives you TS types for free
DB	SQLite	Zero-setup file database — perfect for MVP; swap to Postgres later with one line
ORM	Prisma	Type-safe DB queries + migrations generated from one schema file
Integration	Axios	HTTP client on the React side; interceptors handle auth + 401 redirects centrally

Repository structure (what you're building toward)

A **monorepo** using npm workspaces — one `npm install` wires up both apps, and they can share types.

```

workout/
  package.json          # root: npm workspaces + scripts to run both apps
  atomic-moore-mockup/  # existing mockup – kept as design reference, untouched
  server/               # Express + TS + Prisma API
    prisma/schema.prisma # the single source of truth for the database
    prisma/seed.ts       # inserts the exercise catalog + example program
    .env                 # secrets (DATABASE_URL, JWT_SECRET, GOOGLE_* ...)
    src/
      index.ts           # boots the Express app
      env.ts             # zod-validates process.env so missing config fails loudly
      prisma.ts          # one shared PrismaClient
      middleware/auth.ts  # verifies JWT → attaches req.user
      middleware/validate.ts # runs a Zod schema against the request
      modules/<name>/     # each feature = routes + controller + service + schema
        auth/ users/ programs/ exercises/ sessions/ progress/ dashboard/
  client/               # React + Vite + TS + Tailwind
    src/
      main.tsx, App.tsx, router.tsx
      lib/api.ts         # the axios instance (base URL + interceptors)
      lib/queryClient.ts # TanStack Query setup
      auth/AuthContext.tsx # who's logged in; guards routes
      components/        # Button, Input, Card, Pill, SetRow, Modal, BottomNav, Sidebar...
      layouts/AppLayout.tsx # responsive shell: bottom-nav on mobile, sidebar on desktop
      features/<name>/    # one folder per mockup screen group (see build order)
      hooks/             # useSessions, useProgress... (TanStack Query wrappers)

```

Step-by-step setup

Each block is something you'll run. Read the **Why** before the command.

Step 0 — Initialize the monorepo

Why: a root `package.json` with `workspaces` lets `client` and `server` share one install and lets you start both with one command. Git gives you save points from the very first step.

```

cd D:\coding\project\workout
git init
npm init -y
# then edit root package.json: add "workspaces": ["client", "server"]
# and scripts: "dev" runs client+server together (via "concurrently")
npm install -D concurrently

```

Step 1 — Scaffold the React client (Vite + TS)

Why: Vite generates a typed React app with a dev server in seconds — no webpack config to babysit.

```

npm create vite@latest client -- --template react-ts
npm install -w client

```

Step 2 — Add Tailwind to the client

Why: the mockup is utility CSS. Tailwind lets you reproduce it directly in JSX, and its `md:` / `lg:` prefixes are how we get the mobile-first → desktop responsive behavior the mockup shows.

```
npm install -w client -D tailwindcss postcss autoprefixer
npm exec -w client tailwindcss init -p
# then: set content globs in tailwind.config.js, add @tailwind directives to src/index.css,
# and port the mockup's palette (#0f172a primary, slate/gray, green + badge colors) into theme.extend
```

Step 3 — Client libraries (routing, data, http)

Why: React Router = page navigation; TanStack Query = server-state caching/loading/errors; Axios = the HTTP client these hooks call through.

```
npm install -w client react-router-dom @tanstack/react-query axios
```

Step 4 — Scaffold the Express server (TS)

Why: Express is the API. `tsx` runs TypeScript directly in dev with hot reload (no build step while developing); the typed deps give you autocompletion and safety.

```
npm init -y -w server
npm install -w server express cors cookie-parser zod jsonwebtoken bcryptjs
npm install -w server -D typescript tsx @types/express @types/cors @types/cookie-parser @types/jsonwebtoken @types/bcryptjs
npm exec -w server tsc --init
# add server script "dev": "tsx watch src/index.ts"
```

Step 5 — Add Prisma + SQLite

Why: Prisma turns one `schema.prisma` file into a typed DB client and versioned migrations. SQLite means the database is just a local file — nothing to install or run.

```
npm install -w server @prisma/client
npm install -w server -D prisma
npm exec -w server prisma init --datasource-provider sqlite
# write the models (next section) into prisma/schema.prisma, then:
npm exec -w server prisma migrate dev --name init # creates the DB + tables
```

Step 6 — Google OAuth deps

Why: Passport handles the Google sign-in handshake; after Google verifies the user we issue our **own** JWT, so the rest of the API treats Google and email/password users identically.

```
npm install -w server passport passport-google-oauth20
npm install -w server -D @types/passport @types/passport-google-oauth20
# You'll create a Google OAuth Client ID in console.cloud.google.com and put the
# client id/secret + callback URL into server/.env (do email/password first; wire Google after).
```

Step 7 — Seed the database

Why: a fresh account should feel alive. The seed inserts the exercise catalog (grouped by muscle, which powers the swap modal) and an example program with the 5 day-types, so you can edit instead of starting from a blank screen.

```
# add prisma.seed config + a "seed" script, then:
npm exec -w server prisma db seed
```

Step 8 — Run it

Why: confirm both halves talk to each other before building features on top.

```
npm run dev          # root script starts client (Vite) + server (tsx) together
```

Data model (`prisma/schema.prisma` , SQLite)

```
User          id, email(unique), username, passwordHash?(null for Google-only),
               googleId?(unique), avatarInitial, createdAt
Exercise       id, name, category(Chest|Back|Shoulders|Legs|Arms|Core ... ),
               ownerId?(null = global catalog), createdAt          # powers swap-modal groups
Program        id, ownerId, name, isActive, createdAt              # a user's routine
Day            id, programId, name, badgeColor, order              # e.g. "TORSO A"
DayExercise    id, dayId, exerciseId, order                        # exercises planned for a day
Session        id, userId, dayId?, dayNameSnapshot, badgeColorSnapshot,
               startedAt, endedAt?, durationSec?                  # a logged workout
SessionExercise id, sessionId, exerciseId, nameSnapshot, order
SetLog         id, sessionExerciseId, kind(WARMUP|NORMAL|DROP), weightKg, reps, order
```

Why the snapshots? `dayNameSnapshot` / `nameSnapshot` copy the name into the session at log time. If you later rename or delete a routine or exercise, old history still reads correctly. Progress and personal bests are **derived** by querying `SetLog` joined back to `Exercise` — not stored separately, so they're always consistent.

Backend API (Express + TS + Zod)

Every route validates input with a `validate(schema)` middleware (Zod); protected routes use an `auth` middleware that verifies the JWT and sets `req.user`. **Why this shape:** validation and auth live in one place each, so individual handlers stay tiny and you can't forget them.

- **Auth:** `POST /auth/register` , `POST /auth/login` , `GET /auth/me` , `GET /auth/google` + `GET /auth/google/callback` , `POST /auth/logout` .
- **Users:** `PATCH /users/me` , `POST /users/me/password` (current + new, bcrypt).
- **Programs / Days / DayExercises:** full CRUD scoped to owner — backs the routine editor.
- **Exercises:** `GET /exercises?search=&category=` (grouped, for the swap modal), `POST /exercises` .
- **Sessions:** `POST /sessions` (start from a `dayId` → snapshots the day's exercises into the session), `GET /sessions` (history grouped by month), `GET /sessions/:id` (detail), `PATCH /sessions/:id` (finish → set `endedAt` / `durationSec`), `DELETE /sessions/:id` ; within a session: add/remove/swap `SessionExercise` and `POST|PATCH|DELETE SetLog` .
- **Progress:** `GET /progress?exerciseId=` → personal best, per-session top-set history, trend array.
- **Dashboard:** `GET /dashboard` → suggested next day, recent sessions, lifetime stats.

Auth strategy (MVP): short-lived JWT in an **httpOnly cookie** — simplest secure default, and the browser sends it automatically so the client never juggles tokens. Refresh tokens are deferred to the production phase. `bcryptjs` hashes passwords. `env.ts` zod-validates `DATABASE_URL` , `JWT_SECRET` , `GOOGLE_CLIENT_ID/SECRET` , `CLIENT_URL` so the server refuses to start with missing config.

Frontend (React + Vite + TS + Tailwind)

- **Routing:** React Router. Public (`/login` , `/register`) vs. protected (everything else), gated by `AuthContext` , which calls `GET /auth/me` once on load to learn who's signed in.
- **Server state:** TanStack Query over the shared `Axios` instance (`lib/api.ts`). A response interceptor redirects to `/login` on any 401 — so session expiry is handled in one place.
- **Porting the mockup:** its CSS classes map straight to Tailwind utilities. Put the design tokens in `tailwind.config.js` , then build small reusable components (`Button` variants black/green/ghost/google, `Input` , `Card` , `Pill` , `SetRow` , `ExerciseCard` , `Modal` , `BottomNav` , `Sidebar`) so each page stays thin.
- **Pages** map 1:1 to the mockup screens, **plus** a minimal routine editor (program → days → exercises) to deliver "user-editable", reusing the same components.

Mobile-responsive priority (the core requirement)

- **Mobile-first:** base Tailwind classes target the phone layout from the mockup; `md:` / `lg:` layer on the desktop two-panel + sidebar variants the mockup also includes.
 - `AppLayout` renders the bottom tab nav on small screens and the left sidebar from `lg:` up — one set of nav items, two presentations.
 - **Active Workout** is the make-or-break screen: stacked accordion cards on mobile; desktop two-panel (exercise list left, sets right) at `lg:` . Workout + rest timers run client-side (`useWorkoutTimer`); set edits persist via mutations as you log them.
 - Test at **375px** (mockup phone width), a mid tablet, and **≥1080px** (mockup desktop width).
-

Build order (MVP milestones)

Do these in order — each one is demoable before moving on.

1. **Scaffold** (Steps 0–8 above): both apps run, DB migrated + seeded.
 2. **Auth end-to-end:** register/login (email/pw) → JWT cookie → `AuthContext` → protected routing → profile + change-password. Add **Google OAuth** once email/pw works.
 3. **Catalog + routines:** exercises endpoint + seed; program/day/dayExercise CRUD + minimal editor UI.
 4. **Active Workout (core):** start session → log/edit/delete sets, warm-up/drop kinds, swap modal, add/remove exercise, timers, finish → summary.
 5. **History + Session Detail:** month-grouped list → read-only detail tables.
 6. **Progress + Dashboard:** per-exercise PB/history/trend; dashboard suggestion + recents + stats.
 7. **Responsive polish** across all breakpoints; empty/loading/error states.
-

Path to production (after the MVP is validated)

Refresh-token rotation; Vitest (client) + Supertest (API) tests on auth/sessions/progress; GitHub Actions CI; Dockerfile + compose; switch SQLite→Postgres by changing the Prisma datasource; add `helmet` , rate-limiting, tightened CORS; error monitoring.

Verification

- **Backend:** `npm run dev`, then hit routes with a REST client / curl. Confirm Zod rejects bad input (400) and `auth` blocks unauthenticated requests (401). Run `npm exec -w server prisma studio` to eyeball the seeded data.
- **Full flow (manual):** register → see the seeded example program on the dashboard → start TORSO A → log sets (including a warm-up and an inline edit) → swap an exercise → finish → confirm it appears in History → open Session Detail → check Progress PB/trend updated → edit profile / change password.
- **Responsive:** DevTools device toolbar at 375px, ~768px, ~1080px — bottom-nav↔sidebar switch and the Active Workout single-panel↔two-panel layout both behave.
- **Google OAuth:** sign in with Google in a clean browser session → lands authenticated with a User row.
- (Production phase) `npm test` green in CI before any deploy.